

Fault Models:

1. CommentCountCheck
 - 1.1. Count multiple "//" on the same line as separate comments
2. CommentLineCountCheck
 - 2.1. Count multiple "//" on the same line as separate lines
 - 2.2. Reset the line number for an initial /* if a nested /* exists following it
3. ExpressionCountCheck
 - 3.1. Fails to count "x = y" as an expression
 - 3.2. Count "100%" as an expression
4. LoopCountCheck
 - 4.1. Counts do-while as 2 loops
5. OperandCountCheck
 - 5.1. Counts int x as both integer and variable tokens
 - 5.2. Counts operands in comments
6. OperatorCountCheck
 - 6.1. Counts a defined variable enclosed with <> as two operators
 - 6.2. Fails to count pairing operators such as {}, [], and ()
 - 6.3. Counts operators in comments
7. HalsteadChecks [All]
 - 7.1. Gets incorrect values for operands/operators
 - 7.2. Counts pairing operators as separate operators

Pitclipse Results:

For this deliverable, I ran pitclipse in my terminal, much like I did the white-box testing, which is why these results look the way they do.

With my white-box testing, I was unable to come up with too many more unit tests for what I had already created, and so mainly the second part of this was testing null values in the normal metrics, since the Halstead ones relied on them, and the only situation I could see going sideways was in the difference.

I was unable to get Mockito to mock the Checkstyle classes, so I was unable to call verification with statements like: `verify(mockAST).beginTree` or `verify(spyCheck).log([parameters])`. Which means that Pitclipse's NO_COVERAGE tests are likely all on my `finishTree` methods.

Also, all branches (at least that I found) were covered in the tests I submitted for Deliverable 2.

No Modifications (Screenshots):

```

=====
- Timings
=====
> pre-scan for mutations : < 1 second
> scan classpath : < 1 second
> coverage and dependency analysis : < 1 second
> build mutation tests : < 1 second
> run mutation analysis : 8 seconds
=====
> Total : 9 seconds
=====
> 0 tests took longer than 2000 ms
> Slowest test ([engine:junit-jupiter]/[class:CatATests.CommentCountTest]/[method:testFinishTree()]) took 460 ms
> Largest test ([engine:junit-jupiter]/[class:CatBTests.HEffortTest]/[method:testGetEffort()]) covered 28 blocks
=====
- Statistics
=====
>> Line Coverage (for mutated classes only): 183/205 (89%)
>> 11 tests examined
>> Generated 94 mutations Killed 80 (85%)
>> Mutations with no coverage 11. Test strength 96%
>> Ran 100 tests (1.06 tests per mutation)

```

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
11	89% 183/205	85% 80/94	96% 80/83

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
CatACheckPackage	6	93% 158/170	89% 54/61	98% 54/55
CatBCheckPackage	5	71% 25/35	79% 26/33	93% 26/28

Report generated by [PIT](#) 1.22.0

Enhanced functionality available at [arcmutate.com](#)

More Unit Testing Included (Screenshots):

```

=====
- Timings
=====
> pre-scan for mutations : < 1 second
> scan classpath : < 1 second
> coverage and dependency analysis : < 1 second
> build mutation tests : < 1 second
> run mutation analysis : 8 seconds
=====
> Total : 9 seconds
=====
> 0 tests took longer than 2000 ms
> Slowest test ([engine:junit-jupiter]/[class:CatATests.CommentCountTest]/[method:testFinishTree()]) took 461 ms
> Largest test ([engine:junit-jupiter]/[class:CatBTests.HEffortTest]/[method:testGetEffort()]) covered 30 blocks
=====
- Statistics
=====
>> Line Coverage (for mutated classes only): 137/159 (86%)
>> 11 tests examined
>> Generated 95 mutations Killed 81 (85%)
>> Mutations with no coverage 11. Test strength 96%
>> Ran 105 tests (1.11 tests per mutation)

```

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
11	86% 137/159	85% 81/95	96% 81/84

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
CatACheckPackage	6	90% 110/122	89% 54/61	98% 54/55
CatBCheckPackage	5	73% 27/37	79% 27/34	93% 27/29

Report generated by [PIT](#) 1.22.0

Enhanced functionality available at arcmutate.com

In the new version of the white-box tests, I also found that my Halstead length check would return NaN for a 0 value of unique operands, so I added an if statement there. This appears to have made the line coverage worse overall, although 46 lines appear to have vanished. I am assuming these missing lines are due to me cleaning up the different Halstead checks for volume and effort by calling my get methods, rather than putting in the whole equation/calls each time. But otherwise, the test strength and mutation coverage remained the same.

Commenting my Unit Tests:

```
=====
- Timings
=====
> pre-scan for mutations : < 1 second
> scan classpath : < 1 second
> coverage and dependency analysis : < 1 second
> build mutation tests : < 1 second
> run mutation analysis : 8 seconds
=====
> Total   : 9 seconds
=====
> 0 tests took longer than 2000 ms
> Slowest test ([engine:junit-jupiter]/[class:CatATests.CommentCountTest]/[method:testFinishTree()]) took 469 ms
> Largest test ([engine:junit-jupiter]/[class:CatBTests.HEffortTest]/[method:testFinishTree()]) covered 30 blocks
=====
- Statistics
=====
>> Line Coverage (for mutated classes only): 135/159 (85%)
>> 11 tests examined
>> Generated 95 mutations Killed 73 (77%)
>> Mutations with no coverage 12. Test strength 88%
>> Ran 110 tests (1.16 tests per mutation)
```

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
11	85% 135/159	77% 73/95	88% 73/83

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
CatACheckPackage	6	89% 109/122	87% 53/61	98% 53/54
CatBCheckPackage	5	70% 26/37	59% 20/34	69% 20/29

Report generated by [PIT](#) 1.22.0

Enhanced functionality available at arcmutate.com

This does show a massive discrepancy in the Halstead metrics' mutation coverage, and a slightly lower score all-around for the test strength and line coverage.

Black-Box Improvements:

Initially, my black-box tests detected bugs with the following checks, based on the fault models:
OperatorCheck

- Failed to count = as an operator
- Failed to count any of the {}, [], or () operators

ExpressionCheck

- Ignored = as an assignment expression

CommentLineCheck

- Did reset the line value with nested /* symbols
- Also counted multiple // on the same line

CommentCountCheck

- Counted multiple // on the same line

OperandCount

- Class/method/package names were counted as operands

After finding these issues, I fixed or addressed all of them except for the operator check's issue with pairing operators. I ran out of time and was unable to find a way to distinctly separate the operators within the code from the operators in things like the method/class declaration, or the brackets surrounding the entirety of the class.

For further tests, if I had the time, I would've liked to fix and address the issues of pairing operators, failing to count the variable definitions in my operand count, with things like int x; rather than int x = some value. I also would've liked to address the CommentCountCheck, as I have no logic in it for addressing when there is a comment such as: // this /* all in one line.

However, it's 3 am. And I do not think I am salvaging my grade here.

Also, I tested all of these through my terminal as well, which is why my screenshots still look so... janky. I've got not a singular clue how you wanted us to run these and I totally lost the ability to do coverage, so honestly, I know this is terrible, this is all I could figure out. If I see you next time this class is offered, I swear I'll do at least somewhat better. :(

Class Testing:

Class testing would create less test cases in this instance, although the length of them may make up for the smaller number of overall tests. Instead of reviewing each function, class testing would also allow each test to invoke/call each method within the class directly, rather than through numerous test cases with mocked versions, leading to less verification on the cross-method references. But, because most of the methods in the Checkstyle class are overridden, each test would still need to ensure that each of these overridden methods are called. So, we'd get to eliminate the separation of the method tests and the mocking, but would still need to call almost every method within the new overarching class test.

Also, with class testing, we'd gain the ability to test for state-dependent faults; the transitions between the different methods within each check class would be able to be tested for potential errors upon execution in technically legal, but overall incorrect series, which is something we couldn't do before with white-box testing. Multiple variations of function orders could be tested, rather than a linear or hand-chosen set, although through utilizing data slices, this could be a shared fault between class and unit testing.

Since most of the focus of class testing seems to be on the state of the class and the transitions therein, in the case of these checks, I believe that the biggest overall difference would be the consolidation of tests, and the secondary difference would be attempting to find faults in the order and frequency of the beginTree, visitToken, and finishTree methods. The focus of the tests would shift from just validating the final output to figuring out if the mentioned methods correctly updated and accessed the output.

README

I'm not certain if I entirely missed the mark on Deliverable 2, or if this wasn't the summary you wanted, but I did have the lower section of this (marked with ****Deliverable 2****) in my README. What I threw in there for this deliverable is also pasted below. It includes a brief description of the checks and their tests, as well as the assumptions I made.

I have no idea if these are wildly off the mark or not what you wanted, but I do have them. The format looks better on GitHub, but they're here as well now too.

****Deliverable 3:****

****CommentCountCheck****

 Code: Gets the total number of // and /* tokens.

> I assumed all comments started with /* would finish, and count as a comment. I also assumed there would be no comments such as:

> // example // I like adding these //

Tests: Methods with return values are evaluated for if the return exactly matches the expected value, and void methods are tested for if the class values are updated correctly.

****CommentLineCountCheck****

 Code: Gets the total number of // and the number of lines between /* and */ tokens.

> In this, I also assumed there would be no comments like above, or like:

> /*

> /* Look, some text

> * Still going */

> * Continuing on */

Tests: Methods with return values are evaluated for if the return exactly matches the expected value, and void methods are tested for if the class values are updated correctly. All variations of TokenType with unique updating methods are tested as well.

****ExpressionCountCheck****

 Code: Gets all lines with at least one of: +, -, *, %, /; each line is counted as 1, no matter how many of these characters are in it.

> I assumed that an expression would be kept on one line and only use basic arithmetic values in it. I did not count something like i += 1 as an expression.

Tests: Methods with return values are evaluated for if the return exactly matches the expected value, and void methods are tested for if the class values are updated correctly.

****LoopCountCheck****

 Code: Gets all tokens with the text "for", "foreach", "while", and "do while".

> For the tokens here, my assumption was that only the for/while loops would count as loops, and something that referred back to itself didn't.

Tests: Methods with return values are evaluated for if the return exactly matches the expected value, and void methods are tested for if the class values are updated correctly.

****OperandCountCheck****

 Code: Gets all tokens that are variables, doubles, floats, integers, strings, and true/false. Also, records the amount of unique characters from that list, by comparing each new token to previously found ones.

> This metric at least covers the basic examples of an operand, as something I assumed could be edited or throwing into an equivalency (aside from classes) or equation.

Tests: Methods with return values are evaluated for if the return exactly matches the expected value, and void methods are tested for if the class values are updated correctly. Differing tokens are tested to find unique counts and non-unique counts.

****OperatorCountCheck****

Code: Gets all tokens included in: +, -, *, /, %, ++, --, ==, !=, >>>, <<<, >, <, >=, <=, ^, |, ||, &&, ^=, +=, -=, /=, *=, %=, >>>=, |=, ||=. Also, records the amount of unique characters from that list, by comparing each new token to previously found ones.

> For the operators, this list isn't exhaustive, but I defined the operators as anything that either was part of an equation, or did something to edit or change some previous value. Also, I assumed things like () and [] weren't operators, and only used arithmetic, logical, and bitwise operators in this list.

Tests: Methods with return values are evaluated for if the return exactly matches the expected value, and void methods are tested for if the class values are updated correctly. Differing tokens are tested to find unique counts and non-unique counts.

****HalsteadChecks****

Code: Each Halstead Check differs only in their methods for getting/calculating the metric values; the Halstead length gets the total amount of operands and operators, the vocabulary is the total number of unique operators and operands, the volume is the Halstead length * $\log_2(\text{vocabulary})$, the difficulty is $(0.5(\text{unique operators}) * \text{total operators}) / \text{unique operators}$, and the effort is the difficulty * the volume.

> The operands and operators were calculated through the variables from the Operand/Operator checks.

Tests: Each of these checks had simple tests, that confirmed their implemented getAcceptableTokens(), getRequiredTokens(), and getDefaultTokens() all returned their Token arrays ({0}) as expected. They also had various finishTree() and getMethod() tests that evaluated if mock values calculated correctly, and were updated correctly.

****Deliverable 2:****

Test results:

70/70 tests ran, completed, and came back positive/without failure or error.

Explanation of Incomplete Coverage:

I am not entirely certain what I did on eclipse, my POM.xml file matches the one given, and the DetailAST class is

able to be mocked, but I was unable to mock any of my Checks. Every time, I would get an error such as:

- > [ERROR] Errors:
- > [ERROR] CommentLineCountTest.testFinishTree:140 Mockito
- > Mockito cannot mock this class: class CatACheckPackage.CommentLineCountCheck.

I assume this is because these classes are abstract, although I am uncertain, because in theory they should work.

I threw a comment on each of the checks at the top explaining this; because of this inability to mock the classes, I was unable to get coverage of the visitToken, beginTree, and finishTree methods. I did however run through each if/else branches on the visitToken methods, and ran checks for the expected values of everything within those tests. So the tool doesn't show coverage of that, and I am unable to verify that the values logged are equivalent, but the values themselves are correct on their own.

CommentCountCheck

Overall coverage: 52.8%

Line coverage: 53.8%

Branch coverage: 0/0 branches covered

CommentLineCountCheck

Overall coverage: 33.7%

Line coverage: 33.3%

Branch coverage: 3t 3f/3 branches covered

ExpressionCountCheck

Overall coverage: 47.3%

Line coverage: 33.3%

Branch coverage: 1t 1f/1 branches covered

LoopCountCheck

Overall coverage: 55.6%

Line coverage: 41.7%

Branch coverage: 0/0 branches covered

OperandCountCheck

Overall coverage: 40.1%

Line coverage: 41.9%

Branch coverage: 2t 2f/2 branches covered

OperatorCountCheck

Overall coverage: 78.3%

Line coverage: 46.7%

Branch coverage: 3t 3f/3 branches covered

HalsteadLengthCheck

Overall coverage: 61.8%

Line coverage: 71.4%

Branch coverage: 0/0 branches covered

HalsteadVocabularyCheck

Overall coverage: 61.8%

Line coverage: 71.4%

Branch coverage: 0/0 branches covered

HalsteadDifficultyCheck

Overall coverage: 67.5%

Line coverage: 71.4%

Branch coverage: 0/0 branches covered

HalsteadEffortCheck

Overall coverage: 60.6%

Line coverage: 71.4%

Branch coverage: 0/0 branches covered

HalsteadVolumeCheck

Overall coverage: 69.8%

Line coverage: 71.4%

Branch coverage: 0/0 branches covered